Database

Java Developer Class

Database

Java Basic



What is Database?

Why we need it?

How they works?

Database Models

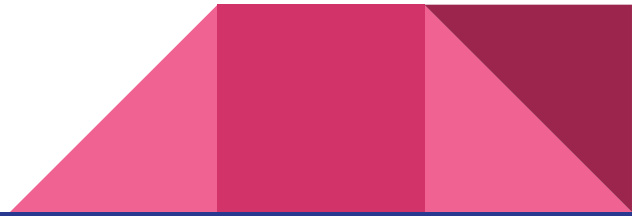
Relational Data Model

What is Database?

An organized collection of structured information, or data

A database is usually controlled by a database management system (DBMS).

The data can then be easily accessed, managed, modified, updated, controlled, and organized.



Data Vs Information

Data	Information
unorganised and unrefined facts	organised data presented in a meaningful context
raw materials which do not carry any specific meaning	a group of data that collectively carries a logical meaning
doesn’t depend on information	depends on data
measured in bits and bytes	is measured in meaningful units like time, quantity, etc
insufficient for decision making	sufficient for decision making

Why we need Database?

We use applications

We put data to application, and also we take data from application

Think about it, if application can't keep data anywhere



We will lose our precious data per each reboot

Where can we save data?

File System?

- Read all data every time
- Difficult to search
- Difficult to update
- If data grow much, Oh No!

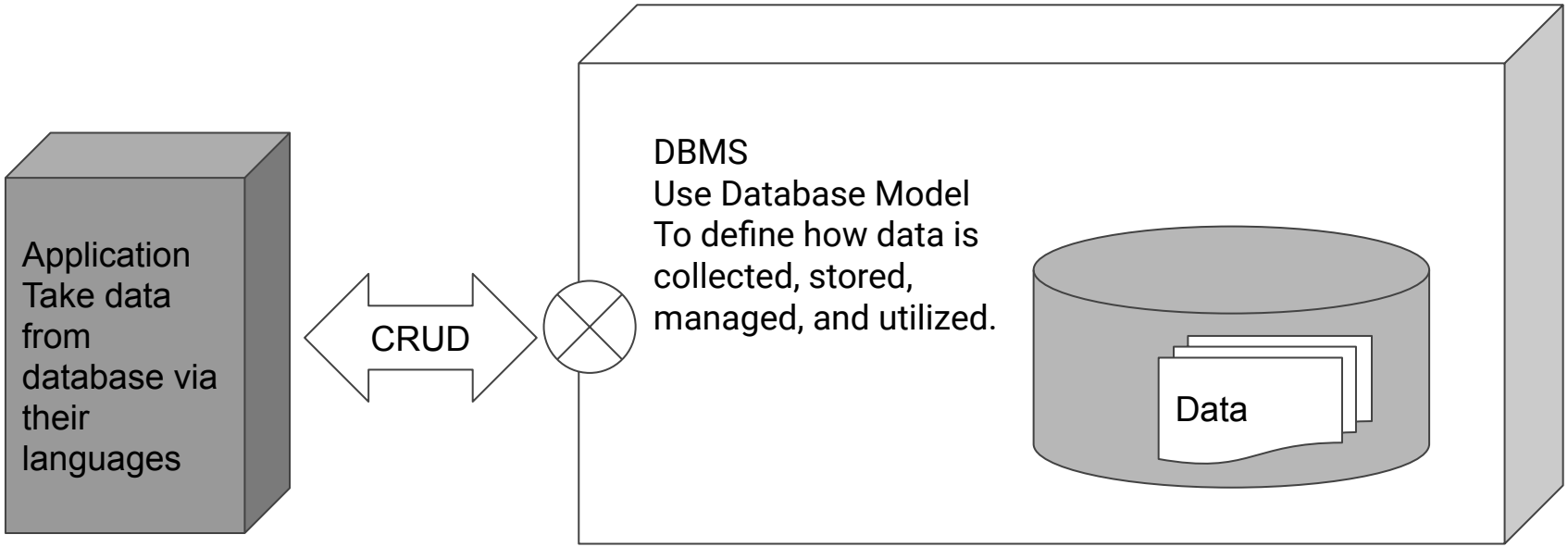
Database is the solution

- Manage Separately from application
- Easy to Manipulate

File is not enough!

Feature	Brief Description
Redundancy	Copy of same data in many place
Inconsistency	Copy of some data doesn’t match each other
Hard to access	Really hard and have to implement search algorithm
Unauthorized access	Very hard to control access level to file data
No Concurrent access	Doesn’t allow fully concurrent access to same file.
No backup and recovery	Does not incorporate any backup and recovery of data if a file is lost or corrupted.

How they works?



Database Management System

An interface between the database and its end users or programs

Allow users to retrieve, update, and manage how the information is organized and optimized.

Use database model how to store, retrieve and so on.

Also facilitates oversight and control of databases, enabling a variety of administrative operations such as performance monitoring, tuning, and backup and recovery.

Database Model

A database model shows the logical structure of a database, including the relationships and constraints that determine how data can be stored and accessed.

Individual database models are designed based on the rules and concepts of whichever broader data model the designers adopt.

Most data models can be represented by an accompanying database diagram.



Evolution of the Database

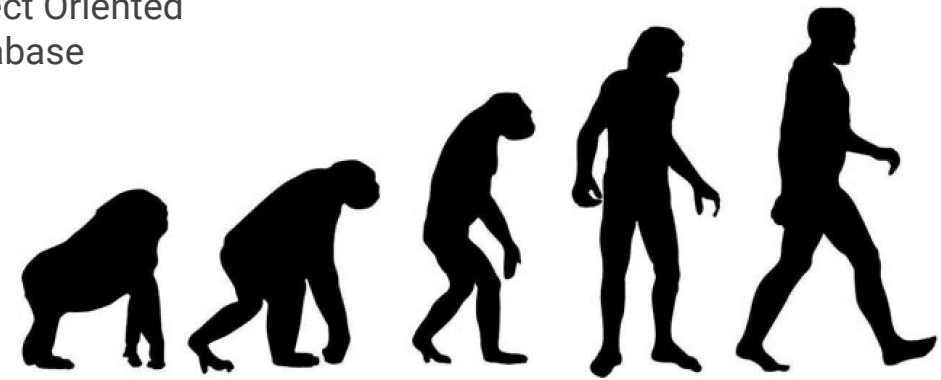
In these days
Cloud Database
Self Driving Database

2009
NoSQL Database

1990
Object Oriented
Database

1980
Relational Database

1960
Navigational Databases
Hierarchical database
Network database



Relational Model

Database

Concepts

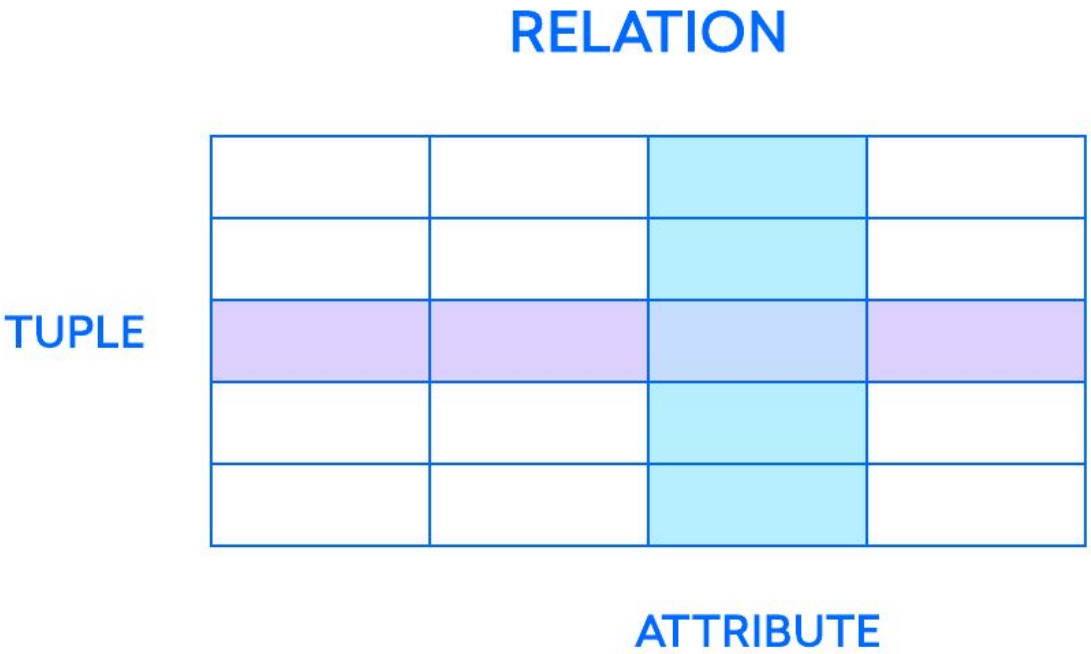
Relationships

Integrity Constraints

Relational Algebra

Good & Bad Things

Relational Model



Relational Model was proposed by E.F. Codd to model data in the form of relations.

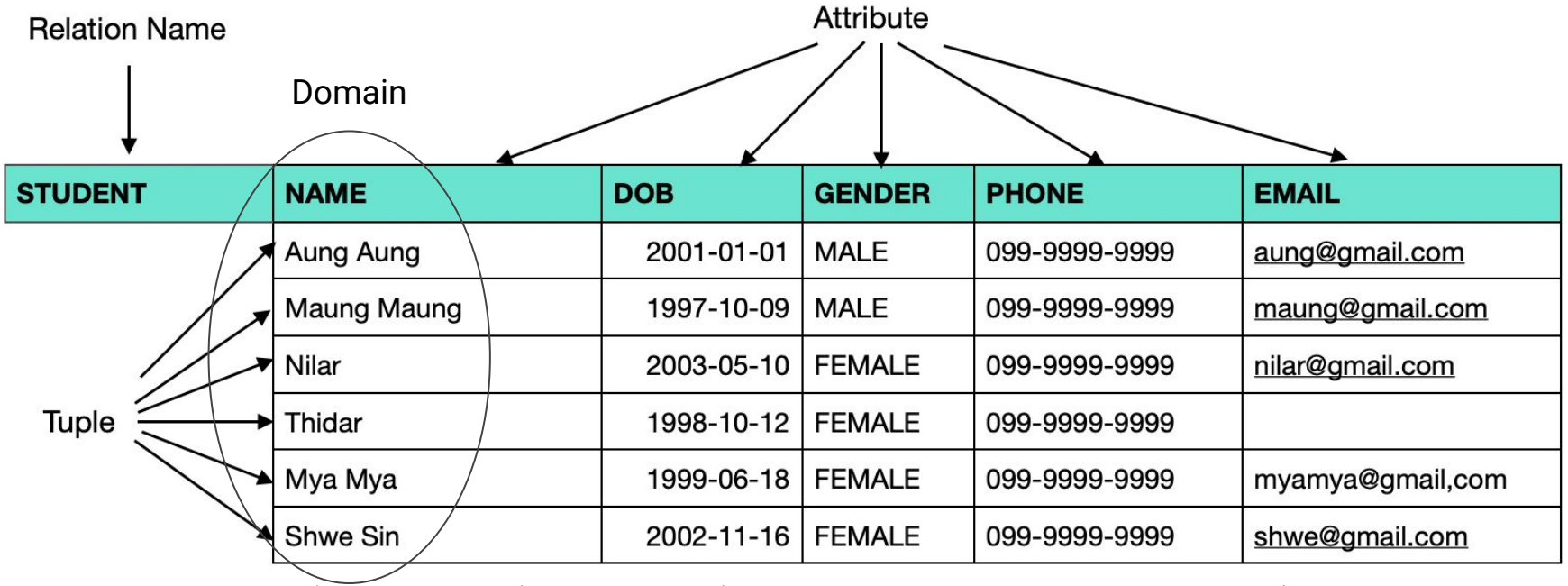
Codd's relational model allowed individual records to be associated with more than one table, thereby enabling "many-to-many" relationships between data points in addition to "one-to-many" relationships.

Codd proposed a language for managing relational data, known as Alpha, which influenced the development of later database languages. (Origin of SQL)

Concepts of Relational Model

Concept	Description
Domain	A set of atomic value that an attribute can take. Value of a column.
Attribute	Properties that define a relation. Name of column in a table.
Tuple	A set of attribute. A row of table that contain a record.
Relation Instance	A finite set of tuples and doesn’t allow duplication. Table in database.
Relation Schema	Define name of relation and attribute of relation. Table definition.
Key	Attributes, that can identify turple in a relation uniquely.
Degree	Number of attributes in a relation
Cardinality	Number of tuples in a relation

Concept of Relation



STUDENT	NAME	DOB	GENDER	PHONE	EMAIL
	Aung Aung	2001-01-01	MALE	099-9999-9999	aung@gmail.com
	Maung Maung	1997-10-09	MALE	099-9999-9999	maung@gmail.com
	Nilar	2003-05-10	FEMALE	099-9999-9999	nilar@gmail.com
	Thidar	1998-10-12	FEMALE	099-9999-9999	
	Mya Mya	1999-06-18	FEMALE	099-9999-9999	myamya@gmail,com
	Shwe Sin	2002-11-16	FEMALE	099-9999-9999	shwe@gmail.com

R(A1, A2, ... , An) = STUDENT(NAME, DOB, GENDER, PHONE, EMAIL)

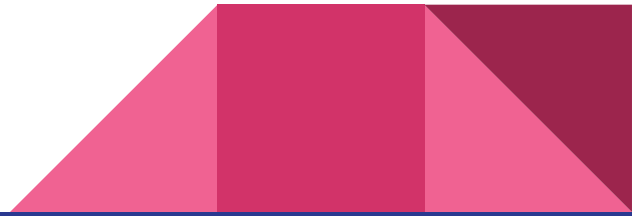
Concept of Keys

Key is an attribute or set of attributes which helps you to identify a tuple(row) in a relation(table).

Keys help you uniquely identify a row in a table by a combination of one or more columns in that table.

Key is also helpful for finding unique record or row from the table.

Key allows you to establish a relationship between and identify the relation between tables.



Types of Key

Key	Description
Super Key	The set of attributes that can uniquely identify a tuple is known as Super Key.
Candidate Key	A minimum set of attributes that uniquely identify tuples in a table. Candidate Key is a super key with no repeated attributes.
Primary Key	A column or group of columns in a table that uniquely identify every row in that table.
Alternate Key	The candidate key other than the primary key is called an alternate key.
Foreign Key	A key that creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity.

Relationships

Relational Model

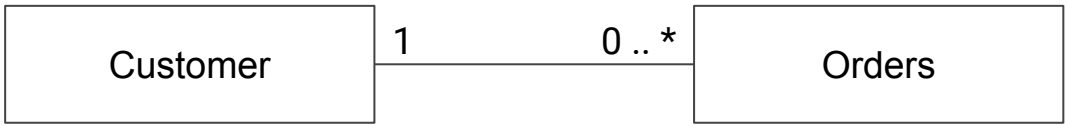
Binary Relationship

1. One to Many
2. One to One
3. Many to Many

Unary Relationship

Ternary Relationship

One to Many (1 : M)

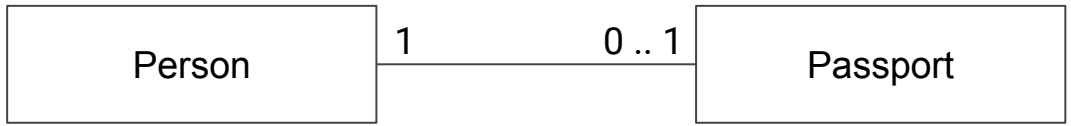


In a one-to-many relationship, one record in a table can be associated with one or more records in another table.

For example a customer can take zero to many orders.

This relationship can be described as many-to-one in other hand.

One to One (1 : 1)



A one to one (1:1) relationship is the relationship of one entity to only one other entity, and vice versa.

It should be rare in any relational database design.

In fact, it could indicate that two entities actually belong in the same table.

Many to Many (M : N)



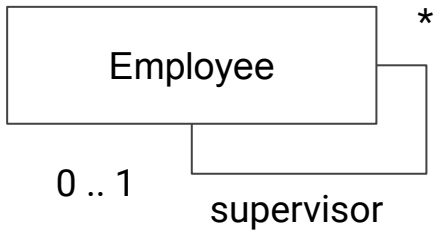
A many-to-many relationship occurs when multiple records in a table are associated with multiple records in another table.

For example, a many-to-many relationship exists between customers and products: customers can purchase various products, and products can be purchased by many customers.

Relational database systems usually don't allow you to implement a direct many-to-many relationship between two tables.

You can break the many-to-many relationship into two one-to-many relationships by using a third table (Orders), called a join table.

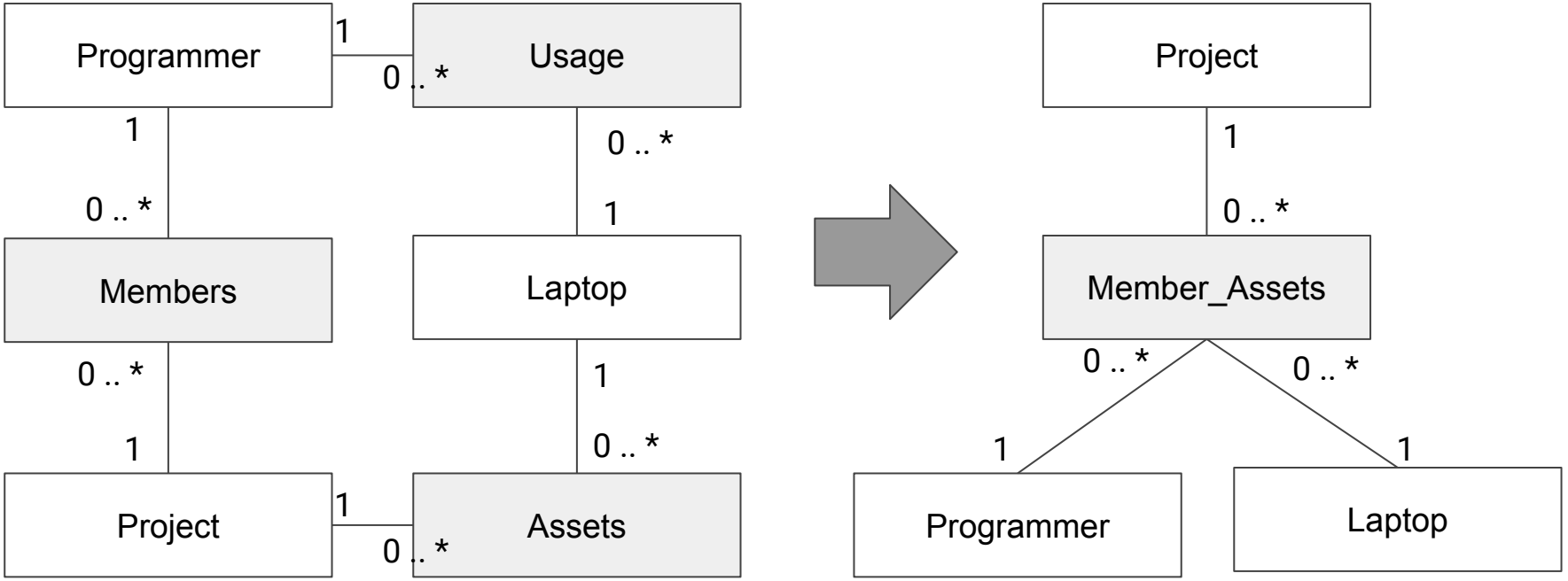
Unary Relationship



A unary relationship, also called recursive, is one in which a relationship exists between occurrences of the same entity set.

In this relationship, the primary and foreign keys are the same, but they represent two entities with different roles.

Ternary Relationship



Relational Integrity Constraints

Relational Model

Relational Integrity constraints in DBMS are referred to conditions which must be present for a valid relation.

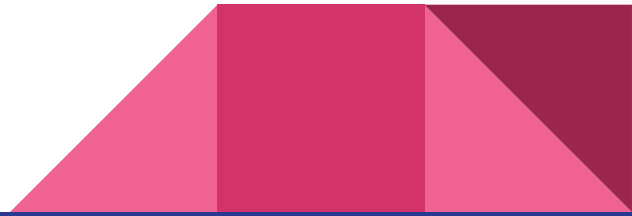
1. Domain Integrity
2. Entity Integrity
3. Referential Integrity

Domain Integrity

Domain integrity specifies that all columns in a relational database must be declared upon a defined domain.

The primary unit of data in the relational data model is the data item. Such data items are said to be non-decomposable or atomic.

Domain Constraint = Data type + Constraints
(NOT NULL / UNIQUE / PRIMARY KEY / FOREIGN KEY / CHECK / DEFAULT)



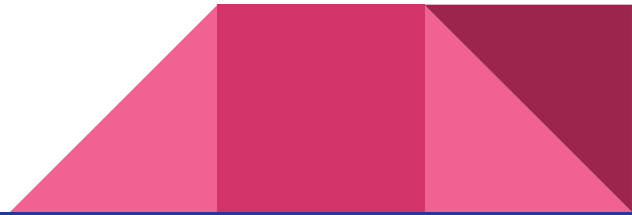
Entity Integrity

To ensure entity integrity, it is required that every table have a primary key.

Neither the PK nor any part of it can contain null values.

This is because null values for the primary key mean we cannot identify some rows.

CHECK Constraints can also be added according to business rules.

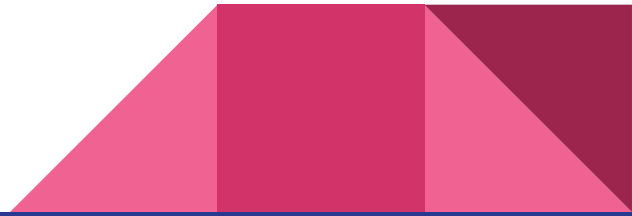


Referential Integrity

Referential integrity requires that a foreign key must have a matching primary key or it must be null.

This constraint is specified between two tables (parent and child); it maintains the correspondence between rows in these tables.

It means the reference from a row in one table to another table must be valid.



Relational Algebra

Relational Model

Basic Operations

Join Operations

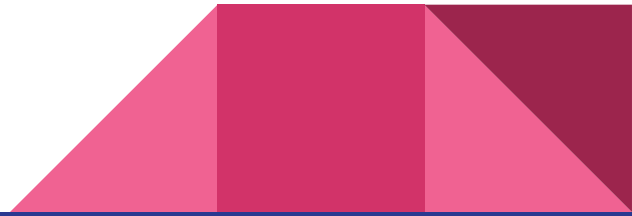
Relational Algebra

A theory that uses algebraic structures with a well-founded semantics for modeling data, and defining queries on it.

The main premise of the relational algebra is to define operators that transform one or more input relations to an output relation.

Primitive theory is introduced by Edgar F. Codd.

This is the base theory of Structured Query Language (SQL)



Basic Operations

Relational Algebra

Relational Operation use Set Theory, but adds additional constraints to these operators.

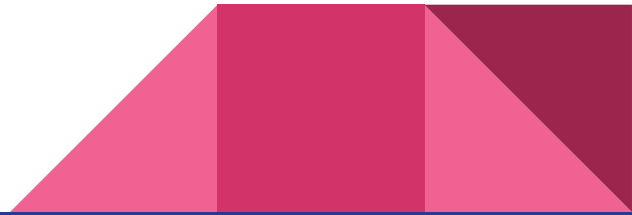
1. Projection
2. Selection
3. Rename
4. Union
5. Difference
6. Intersection
7. Cartesian Product

Projection (Π)

The projection eliminates all attributes of the input relation but those mentioned in the projection list.The projection method defines a relation that contains a vertical subset of Relation.

This operator helps you to keep specific columns from a relation and discards the other columns.

When implemented in SQL standard the "default projection" returns a multiset instead of a set, and the Π projection to eliminate duplicate data is obtained by the addition of the DISTINCT keyword.



Project Operator (Π)

Registration

ID	Name	Date	Course
1	Aung Aung	2021-10-14	Java Basic
2	Maung Maung	2021-10-15	Java Basic
3	Thidar	2021-10-18	Spring
4	Nilar	2021-10-20	Spring

Π name, date (Registration)

Name	Date
Aung Aung	2021-10-14
Maung Maung	2021-10-15
Thidar	2021-10-18
Nilar	2021-10-20

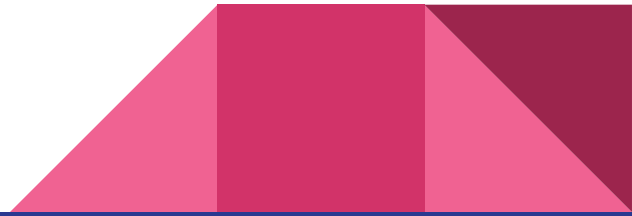
Selection (σ)

The select operation is used for selecting a subset of the tuples according to a given selection condition.

It is used as an expression to choose tuples which meet the selection condition.

Select operator selects tuples that satisfy a given predicate.

A generalized selection is a unary operation and can write the normal selection and the logical operators ∧ (and), ∨ (or) and ¬ (negation).



Select Operator (σ)

Registration

ID	Name	Date
1	Aung Aung	2021-10-14
2	Maung Maung	2021-10-15
3	Thidar	2021-10-18
4	Nilar	2021-10-20

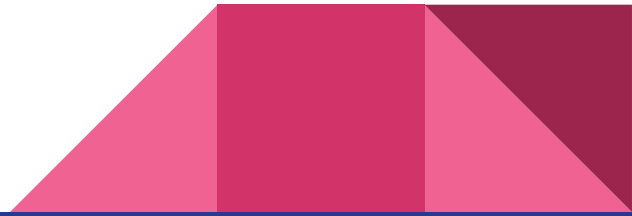
σ date = '2021-10-14' $\vee$ date = '2021-10-15' (Registration)

ID	Name	Date
1	Aung Aung	2021-10-14
2	Maung Maung	2021-10-15

Rename Operation (ρ)

Rename is a unary operation used for renaming attributes of a relation.

ρ (a/b)R will rename the attribute ‘b’ of relation by ‘a’.



Set Theory in Relational Algebra

The relational algebra uses set union, set difference, set intersection and Cartesian product from set theory, but adds additional constraints to these operators.

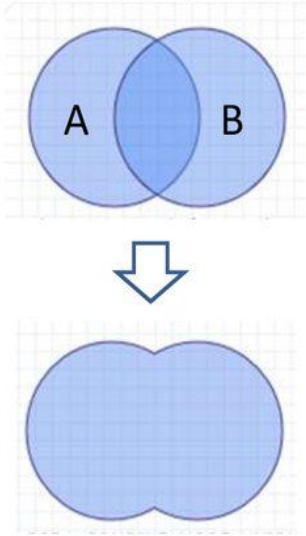
For set union and set difference, the two relations involved must be union-compatible—that is, the two relations must have the same set of attributes.

Because set intersection is defined in terms of set union and set difference, the two relations involved in set intersection must also be union-compatible.

For the Cartesian product to be defined, the two relations involved must have disjoint headers—that is, they must not have a common attribute name.

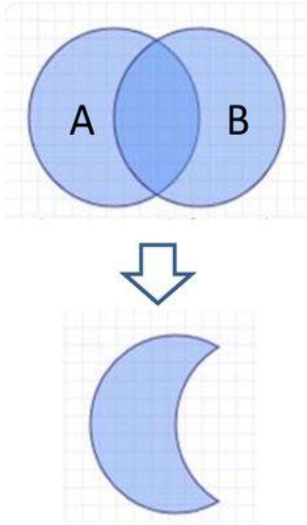
Set Operations

UNION



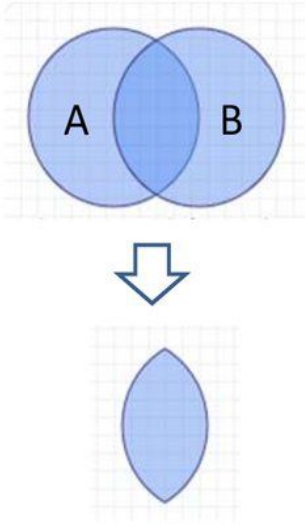
A ∪ B

DIFFERENCE



A - B

INTERSECT



A ∩ B

Union Operator (∪)

R1

Column 1	Column 2	Column 3
1	A	100
2	B	10

R2

Column 1	Column 2	Column 3
2	B	10
3	C	1

R1 ∪ R2

Column 1	Column 2	Column 3
1	A	100
2	B	10
3	C	1

Difference Operator (-)

R1

Column 1	Column 2	Column 3
1	A	100
2	B	10

R2

Column 1	Column 2	Column 3
2	B	10
3	C	1

R1 - R2

Column 1	Column 2	Column 3
1	A	100

Intersect Operator (∩)

R1

Column 1	Column 2	Column 3
1	A	100
2	B	10

R2

Column 1	Column 2	Column 3
2	B	10
3	C	1

R1 ∩ R2

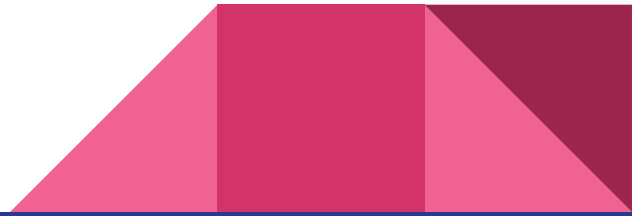
Column 1	Column 2	Column 3
2	B	10

Cartesian Product (×)

For the Cartesian product to be defined, the two relations involved must have disjoint headers—that is, they must not have a common attribute name.

Cartesian product of a set of n-tuples with a set of m-tuples yields a set of "flattened" (n + m)-tuples.

The cardinality of the Cartesian product is the product of the cardinalities of its factors, that is, $|R \times S| = |R| \times |S|$.



Cartesian Product Operator (×)

R1

ID	Name
1	Aung Aung
2	Thidar

R2

Room	Subject
1	Java Basic
2	Web Fundamental

R1 × R2

ID	Name	Room	Subject
1	Aung Aung	1	Java Basic
1	Aung Aung	2	Web Fundamental
2	Thidar	1	Java Basic
2	Thidar	2	Web Fundamental

Join Operations

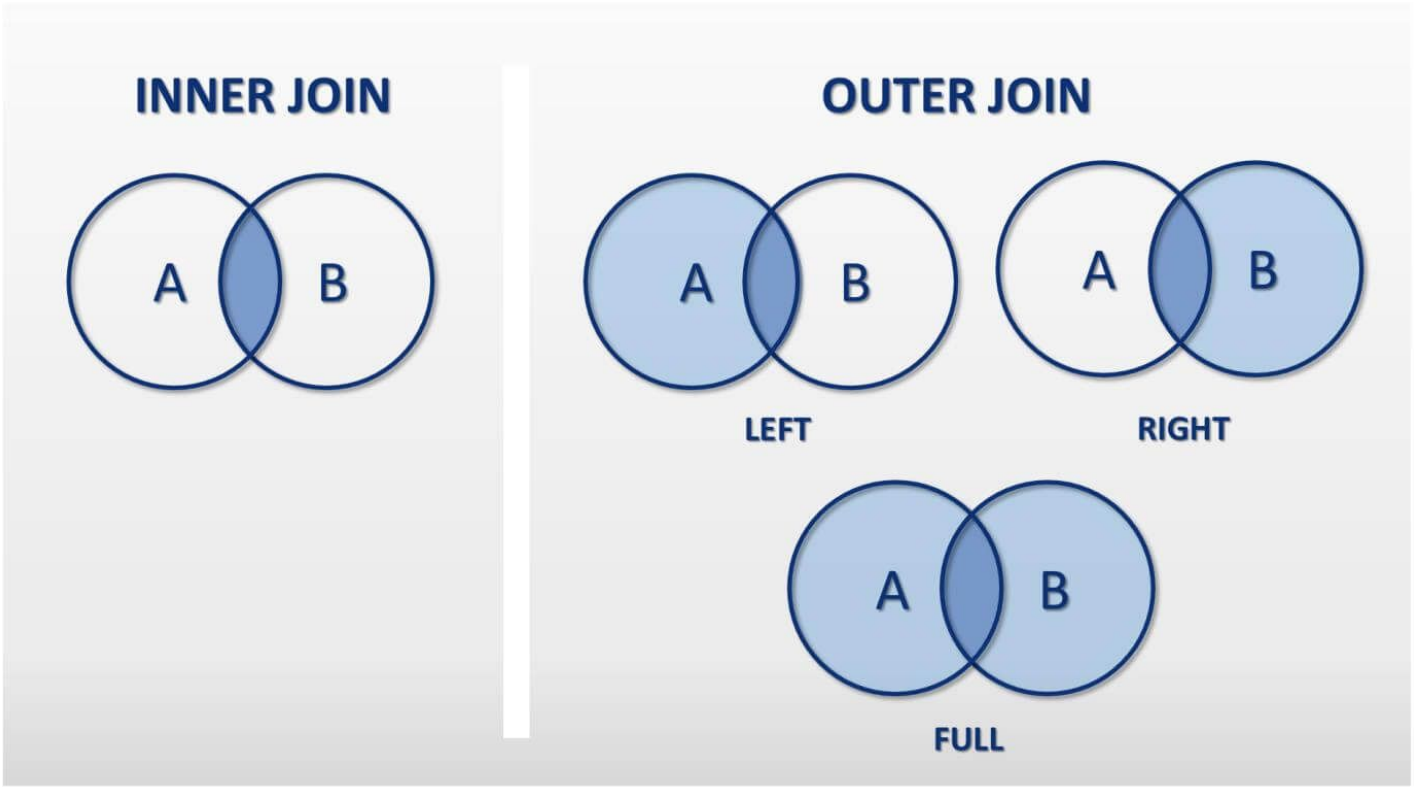
Relational Algebra

Join operation is essentially a cartesian product followed by a selection criterion.

Join operation also allows joining variously related tuples from different relations.

1. Inner Join
2. Outer Join

Join Types



Inner Join

Join Operations

In an inner join, only those tuples that satisfy the matching criteria are included, while the rest are excluded.

1. Natural join
2. Theta join
3. EQUI join

Natural Join (⋈)

SHIP

Location	Shipping
BKK	20
TOKYO	60

AIRLINE

Location	Aircargo
BKK	60
TOKYO	150

SHIP ⋈ (AIRLINE)

Natural join can only be performed if there is a common attribute (column) between the relations. The name and type of the attribute must be same.

If there is no common columns, join the relations as cartesian product.

Location	Fees	Fees
BKK	20	60
TOKYO	60	150

Theta Join ($\bowtie_\theta$)

SHIP

Ship	Shipping
BKK	20
TOKYO	60

AIRLINE

Air	Fees
BKK	60
TOKYO	150

SHIP $\bowtie$ SHIP.Shipping < AIRLINE.Fees (AIRLINE)

The general case of JOIN operation is called a Theta join. It is denoted by symbol θ.

Ship	Shipping	Air	Fees
BKK	20	BKK	60
TOKYO	60	TOKYO	150
BKK	20	TOKYO	150

EQUI Join ($\bowtie_\theta$)

SHIP

Ship	Shipping
BKK	20
TOKYO	60

AIRLINE

Air	Fees
BKK	60
TOKYO	150

SHIP ⋈ SHIP.Ship = AIRLINE.Air (AIRLINE)

When a theta join uses only equivalence condition, it becomes a equi join.

Ship	Fees	Air	Fees
BKK	20	BKK	60
TOKYO	60	TOKYO	150

Outer Join

Join Operations

In an outer join, along with tuples that satisfy the matching criteria, we also include some or all tuples that do not match the criteria.

1. Left Outer Join
2. Right Outer Join
3. Full Join

Left Join (⟕)

SHIP

Ship	Fees
BKK	20
TOKYO	60
LONDON	90

AIRLINE

Air	Fees
BKK	60
TOKYO	150
PARIS	180

SHIP ⟕ SHIP.Ship = AIRLINE.air (AIRLINE)

In the left outer join, operation allows keeping all tuple in the left relation. However, if there is no matching tuple is found in right relation, then the attributes of right relation in the join result are filled with null values.

Ship	Fees	Air	Fees
BKK	20	BKK	60
TOKYO	60	TOKYO	150
LONDON	90	NULL	NULL

Right Join (⟖)

SHIP

Ship	Fees
BKK	20
TOKYO	60
LONDON	90

AIRLINE

Air	Fees
BKK	60
TOKYO	150
PARIS	180

SHIP ⟖ SHIP.Ship = AIRLINE.air (AIRLINE)

In the right outer join, operation allows keeping all tuple in the right relation. However, if there is no matching tuple is found in the left relation, then the attributes of the left relation in the join result are filled with null values.

Ship	Fees	Air	Fees
BKK	20	BKK	60
TOKYO	60	TOKYO	150
NULL	NULL	PARIS	180

Full Join (⟗)

SHIP

Ship	Fees
BKK	20
TOKYO	60
LONDON	90

AIRLINE

Air	Fees
BKK	60
TOKYO	150
PARIS	180

SHIP ⟗ SHIP.Ship = AIRLINE.air (AIRLINE)

In a full outer join, all tuples from both relations are included in the result, irrespective of the matching condition.

Ship	Fees	Air	Fees
BKK	20	BKK	60
TOKYO	60	TOKYO	150
LONDON	90	NULL	NULL
NULL	NULL	PARIS	180

Advantages

- A Relational data model in DBMS is simpler than the hierarchical and network model.
- The relational database is only concerned with data and not with a structure. This can improve the performance of the model.
- The Relational model in DBMS is easy as tables consisting of rows and columns are quite natural and simple to understand.
- It makes possible for a high-level query language like SQL to avoid complex database navigation.
- The Structure of Relational database can be changed without having to change any application.
- Regarding a number of records, or rows, and the number of fields, a database should be enlarged to enhance its usability.

Disadvantages

- Few relational databases have limits on field lengths which can’t be exceeded.
- Relational databases can sometimes become complex as the amount of data grows, and the relations between pieces of data become more complicated.
- Complex relational database systems may lead to isolated databases where the information cannot be shared from one system to another.



THE END

Java Developer Class